

Leah Melamu

ST10477720

Group 02

PROG6212

POE PART 1

Links:.....	3
SECTION A.....	4
Cardinalities explained:.....	5
SECTION B.....	6
Organizer.....	6
Participant.....	11
SECTION C.....	16
Create Statements.....	16
Insert Statements.....	19

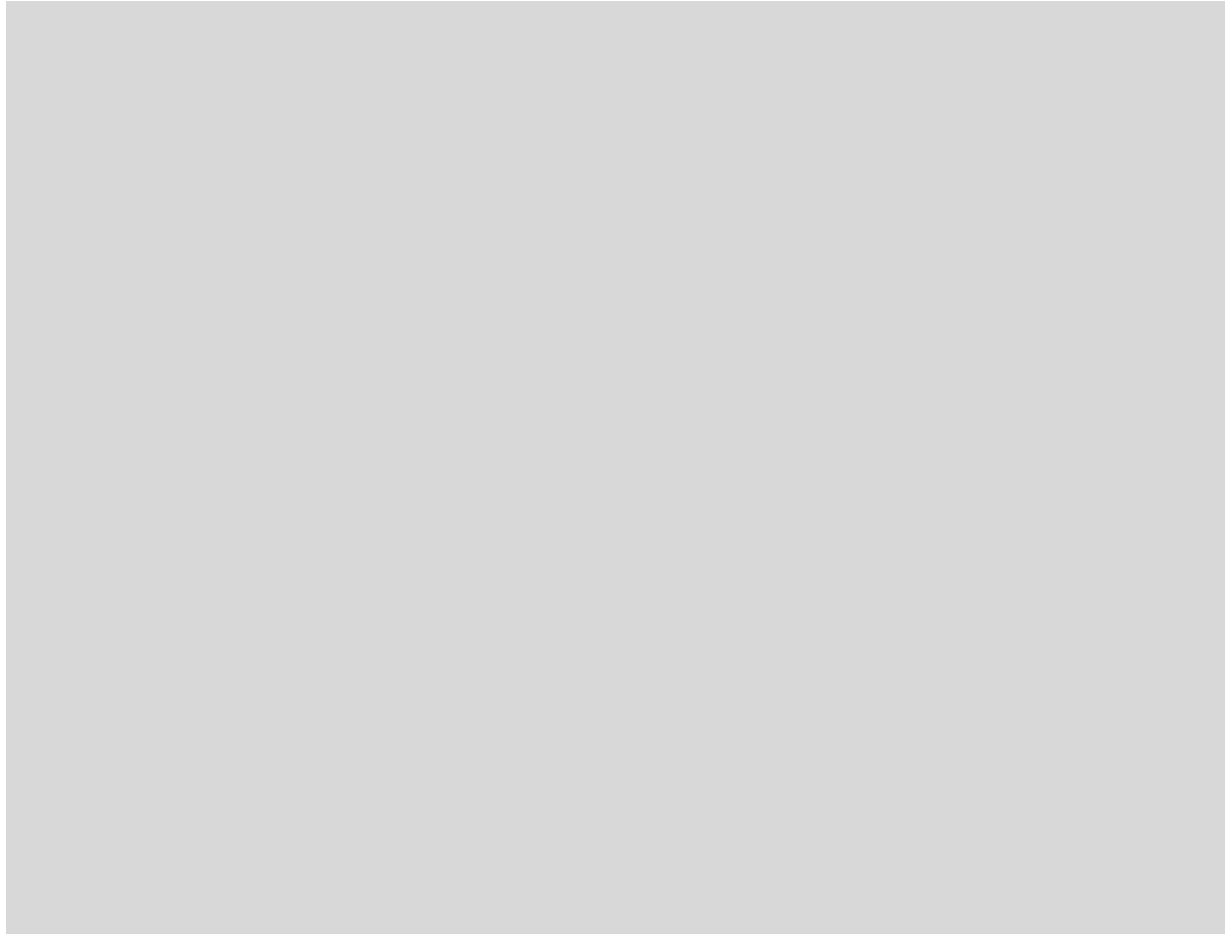
Links:

GitHub link

<https://github.com/wazaaA43/RaceDay-EMS>

YouTube link

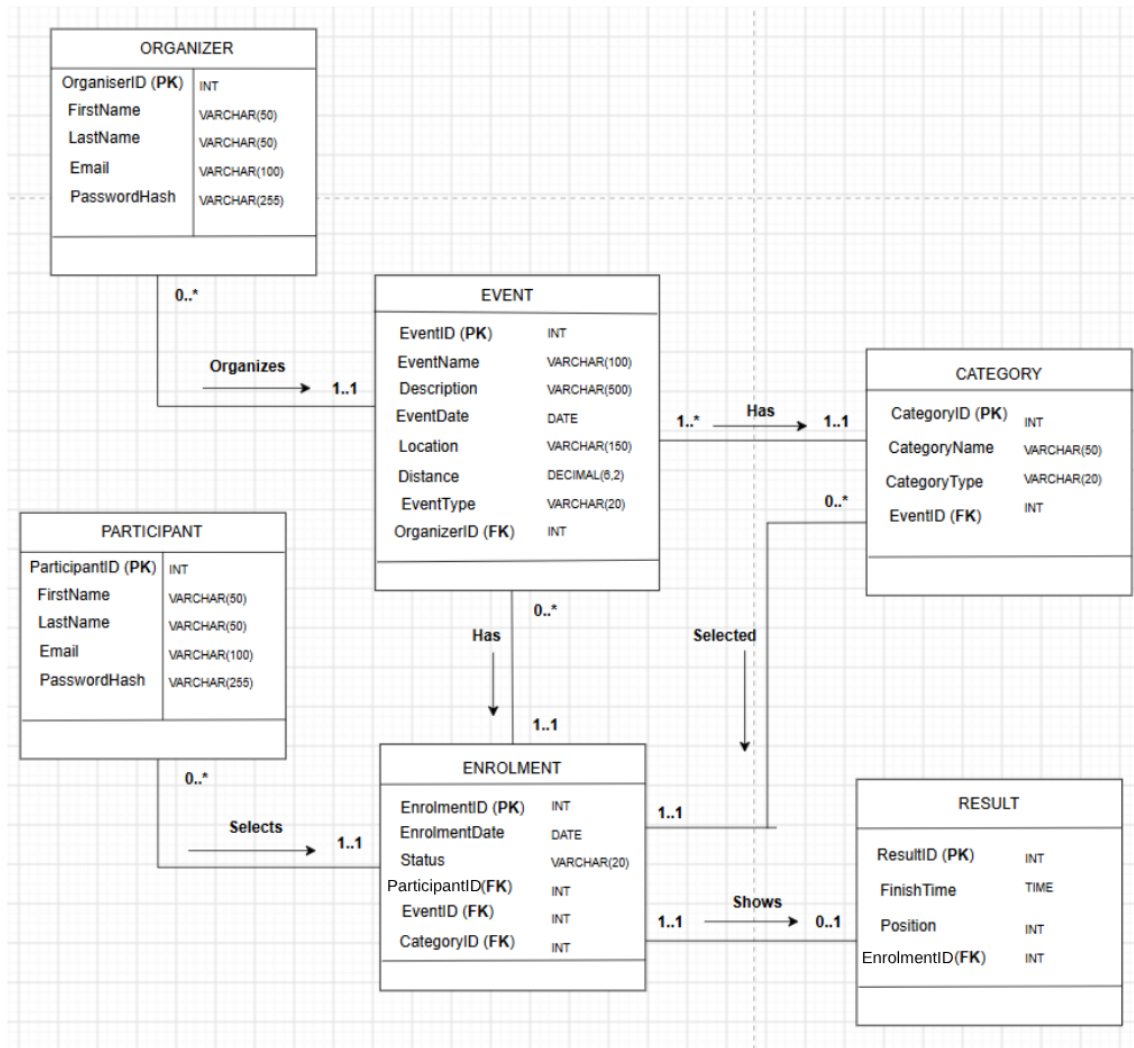
<https://youtu.be/CImXxnzosHM>



If YouTube link doesn't work:

<https://canva.link/nrrv4c96vut8k7o>

SECTION A - Entity Relationship Diagram (ERDs)



The Entity Relationship Diagram (ERD) represents the database structure of the RaceDay Event Management System. Six entities were identified based on the functional requirements: Organizer, Participant, Event, Category, Enrolment, and Result.

The Organizer entity stores information about event organizers. One organizer can manage multiple events, while each event is managed by one organizer. The Event entity stores the event name, description, date, location, distance and event type. EventType is stored as an attribute because the system only requires Run, Walk and Cycle as event types and does not require a separate entity for these values.

The Category entity represents the age or distance categories available for each event. EventID is stored as a foreign key in Category because each category belongs to a specific event.

Participant and Event have a many-to-many relationship because a participant can enter multiple events, and an event can have multiple participants. This relationship is resolved through the Enrolment entity. Enrolment records the participant, event and category selected by the participant.

The Result entity stores the finishing time and finishing position of an enrolment. Result is linked to Enrolment because a result is associated with a participant's participation in a specific event rather than directly with the participant.

Cardinalities explained:

One organizer can organize zero or many events, while each event is organized by exactly one organizer.

One participant can have zero or many enrolments, while each enrolment belongs to exactly one participant.

One event can have zero or many enrolments, while each enrolment belongs to exactly one event.

One event has one or many categories, while each category belongs to exactly one event.

One category can be selected by zero or many participants/enrolments, while each enrolment selects exactly one category.

An enrolment may have no result yet, or one result, while each result belongs to exactly one enrolment.

The many-to-many relationship between PARTICIPANT and EVENT is resolved using the ENROLMENT table. One participant can enroll in zero or many events, while one event can have zero or many participants. Each enrolment records the specific participant, event, and category selected by the participant.

SECTION B - API Endpoint Plan

Organizer

Organizer API Endpoint Plan					
HTTP method	Route	Description	Role required	Request body	Expected response
Authentication:					
1. POST	/api/organizers/register	Registers a new organizer account.	Organizer/ Public	firstName, lastName, email, password,	201 Created -organizer account created; 400 Bad Request -invalid or duplicate details.
2.POST	/api/organizers/login	Authenticates an organizer using their email and password.	Organizer	email, password	200 OK -login successful and authentication token returned; 401 Unauthorized -invalid credentials.
3.POST	/api/organizers/logout	Logs the authenticated organizer out of the system.	Organizer	None	200 OK -logout successful; 401 Unauthorized -authentication required.
User Profile:					

4.GET	/api/organizers/profile	Retrieve s the profile informati on of the authentic ated organize r.	Organizer	None	200 OK - organizer profile returned; 401 Unauthor ized - authentic ation required.
5.PUT	/api/organizers/profile	Updates the authentic ated organize r's profile informati on.	Organizer	firstName, lastName, email	200 OK - profile updated; 400 Bad Request - invalid data.
6.PUT	/api/organizers/passwor d	Allows the organize r to change their passwor d.	Organizer	CurrentPas sword, newPasswo rd	200 OK - password updated; 400 Bad Request - invalid password; 401 Unauthor ized - incorrect current password.
Events:					
7.GET	/api/events	Retrieve s all available events.	Organizer	None	200 OK - list of events returned
8.GET	/api/events/ {eventId}	Retrieve s the details of a specific event	Organizer	None	200 OK - event details returned; 404 Not Found - event does not exist.

9.GET	/api/organizers/events	Retrieve s all events created by the authentic ated organize r.	Organizer	None	200 OK- organizer' s events returned.
10.POS T	/api/events	Creates a new event.	Organizer	eventName, description, eventDate, location, distance, eventType	201 Created- event created; 400 Bad Request- invalid event data.
11.PUT	/api/events/ {eventId}	Updates an event belongin g to the authentic ated organize r.	Organizer	eventName, description, eventDate, location, distance, eventType	200 OK- event updated; 404 Not Found- event does not exist; 403 Forbidde n- organizer does not own the event.
12.DEL ETE	/api/events/ {eventId}	Deletes an event belongin g to the authentic ated organize r.	Organizer	None	204 No Content- event deleted; 404 Not Found- event does not exist.
Categories:					
13.GET	/api/events/ {eventId}/ categories	Retrieve s all categor ie	Organizer	None	200 OK- categor ie s

		s belongin g to an event.			returned; 404 Not Found- event does not exist.
14.POS T	/api/events/ {eventId}/ categories	Creates a new age or distance category for an event.	Organizer	categoryNa me, categoryTy pe	201 Created- category created; 400 Bad Request- invalid category.
15.GET	/api/categories/{category id}	Retrieve s details of a specific category	Organizer	None	200 OK- category returned; 404 Not Found- category does not exist.
16.PUT	/api/categories/{category id}	Updates a category belongin g to the organize r's event.	Organizer	categoryNa me, categoryTy pe	200 OK- category updated; 404 Not Found- category does not exist; 403 Forbidde n- organizer does not manage the event.
17.DEL ETE	/api/categories/{category id}	Deletes a category belongin g to the organize r's event.	Organizer	None	204 No Content- category deleted; 404 Not Found- category does not exist.

Enrolments:					
18.GET	/api/events/{eventId}/enrolments	Retrieve s all participant enrolments for an event belonging to the organizer.	Organizer	None	200 OK -enrolment list returned; 404 Not Found -event does not exist.
19.GET	/api/enrolments/{enrolmentId}	Retrieve s details of a specific participant enrolment.	Organizer	None	200 OK -enrolment details returned; 404 Not Found -enrolment does not exist.
Results:					
20.POST	/api/enrolments/{enrolmentId}/result	Records a participant's finish time and position.	Organizer	finishTime, position	201 Created -result recorded; 400 Bad Request -invalid request; 404 Not Found -enrolment does not exist.

Participant

Participant API Endpoint Plan					
HTTP method	Route	Description	Role required	Request body	Expected response
Authentication:					
1.POST	/api/participants/register	Registers a new participant account.	Participant /Public	firstName, lastName, email, password	201 Created -participant account created; 400 Bad Request -invalid or duplicate details.
2.POST	/api/participants/login	Authenticates a participant using their email and password.	Participant	Email, password	200 OK -login successful and authentication token returned; 401 Unauthorized -invalid credentials.
3.POST	/api/participants/logout	Logs the authenticated participant out of the system.	Participant	None	200 OK -logout successful; 401 Unauthorized -authentication required
User profile:					
4.GET	/api/participants/profile	Retrieves the logged-in participant	Participant	None	200 OK -participant profile returned; 401

		nt's profile.			Unauthorized -authentication required
5.PUT	/api/participants/profile	Updates the logged-in participant's profile information.	Participant	firstName, lastName, email	200 OK -profile updated; 400 Bad Request -invalid data.
6.PUT	/api/participants/password	Allows the participant to change their password.	Participant	CurrentPassword, newPassword	200 OK -password updated; 400 Bad Request -invalid password; 401 Unauthorized -incorrect current password.
Events:					
7.GET	/api/events	Retrieves all available events.	Participant	None	200 OK -list of events returned.
8.GET	/api/events/{eventId}	Retrieves details of a specific event.	Participant	None	200 OK -event details returned; 404 Not Found -event does not exist.
9.GET	/api/events/upcoming	Retrieves upcoming events	Participant	None	200 OK -upcoming events returned.

		that participants can enter.			
10.GE T	/api/events/past	Retrieves past events.	Participant	None	200 OK -past events returned.
Categories:					
11.GE T	/api/events/{eventId}/categories	Retrieves the available categories for a specific event.	Participant	None	200 OK -categories returned; 404 Not Found -event does not exist.
12.GE T	/api/categories/{categoryId}	Retrieves details of a specific event category.	Participant	None	200 OK -category details returned; 404 Not Found -category does not exist.
Enrolments:					
13.PO ST	/api/events/{eventId}/enrolments	Enrols the participant in an event using a selected category.	Participant	categoryId	201 Created -enrolment created; 400 Bad Request -invalid category or duplicate enrolment.
14.GE T	/api/participants/enrolments	Retrieves all events in which the authenticated participant	Participant	None	200 OK -participant's enrolments returned.

		nt is enrolled.			
15.GET	/api/participants/enrolments/{enrolmentId}	Retrieves details of one of the participants' enrolments.	Participant	None	200 OK - enrolment returned; 404 Not Found - enrolment does not exist.
16.PUT	/api/participants/enrolment/{enrolmentId}	Allows the participant to change the category selected for their environment.	Participant	categoryId	200 OK - enrolment updated; 400 Bad Request - invalid category; 404 Not Found - enrolment does not exist.
17.DELETE	/api/participants/enrolment/{enrolmentId}	Cancels the participant's own event enrolment.	Participant	None	204 No Content - enrolment cancelled/deleted; 404 Not Found - enrolment does not exist.
Results:					
18.GET	/api/participants/results	Retrieves all of the participants' own results.	Participant	None	200 OK - participant results returned.
19.GET	/api/participants/results/{resultId}	Retrieves one result belonging to the authenti	Participant	None	200 OK - result returned; 404 Not Found -

		cated participant.			result does not exist.
20.GE T	/api/participants/events/{eventId}/result	Retrieves the authenticated participant's result for a specific event.	Participant	None	200 OK - result returned; 404 Not Found - result does not exist.

The API endpoint plan translates the system's functional requirements into operations that can be performed by the application. The endpoints were divided into separate Organizer and Participant tables because each role has different permissions and responsibilities.

The API plan includes authentication endpoints for registration and login, allowing the system to distinguish between organizers and participants. Profile endpoints allow both roles to view and update their own information.

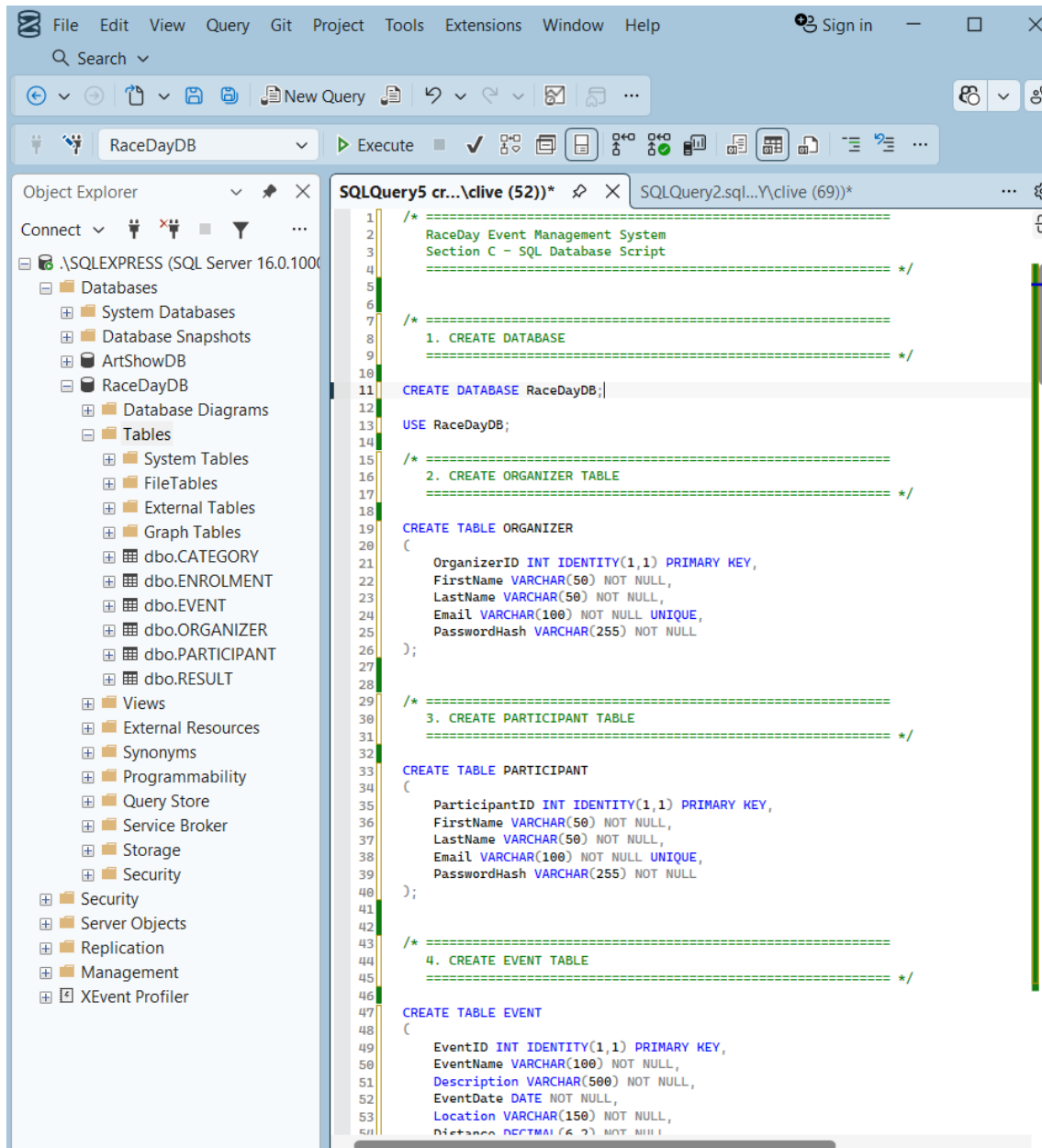
Event endpoints allow organizers to create, update, and delete events, while both roles can retrieve event information. Category endpoints allow organizers to create and manage age or distance categories and allow participants to view available categories.

Enrolment endpoints allow participants to enter events and select a category. The enrolment records the relationship between the participant, event, and selected category. Organizers can retrieve enrolments for events that they manage.

Result endpoints allow organizers to record finishing times and positions, while participants can retrieve their own results. These endpoints collectively provide the functionality required by the system's functional requirements.

SECTION C - SQL Database Script

Create Statements



The screenshot displays the SQL Server Enterprise Manager interface. On the left, the Object Explorer shows the 'RaceDayDB' database selected under the 'Databases' folder. The main pane shows the SQL Query Editor with a script titled 'SQLQuery5 cr... \cdrive (52))*'. The script contains the following SQL statements:

```
1  /* =====
2  RaceDay Event Management System
3  Section C - SQL Database Script
4  ===== */
5
6
7  /* =====
8  1. CREATE DATABASE
9  ===== */
10
11 CREATE DATABASE RaceDayDB;
12
13 USE RaceDayDB;
14
15 /* =====
16 2. CREATE ORGANIZER TABLE
17 ===== */
18
19 CREATE TABLE ORGANIZER
20 (
21     OrganizerID INT IDENTITY(1,1) PRIMARY KEY,
22     FirstName VARCHAR(50) NOT NULL,
23     LastName VARCHAR(50) NOT NULL,
24     Email VARCHAR(100) NOT NULL UNIQUE,
25     PasswordHash VARCHAR(255) NOT NULL
26 );
27
28
29 /* =====
30 3. CREATE PARTICIPANT TABLE
31 ===== */
32
33 CREATE TABLE PARTICIPANT
34 (
35     ParticipantID INT IDENTITY(1,1) PRIMARY KEY,
36     FirstName VARCHAR(50) NOT NULL,
37     LastName VARCHAR(50) NOT NULL,
38     Email VARCHAR(100) NOT NULL UNIQUE,
39     PasswordHash VARCHAR(255) NOT NULL
40 );
41
42
43 /* =====
44 4. CREATE EVENT TABLE
45 ===== */
46
47 CREATE TABLE EVENT
48 (
49     EventID INT IDENTITY(1,1) PRIMARY KEY,
50     EventName VARCHAR(100) NOT NULL,
51     Description VARCHAR(500) NOT NULL,
52     EventDate DATE NOT NULL,
53     Location VARCHAR(150) NOT NULL,
54     Distance DECIMAL(6,2) NOT NULL
```

Object Explorer

Connect

SQL Server 16.0.1000

- Databases
 - System Databases
 - Database Snapshots
 - ArtShowDB
 - RaceDayDB
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.CATEGORY
 - dbo.ENROLMENT
 - dbo.EVENT
 - dbo.ORGANIZER
 - dbo.PARTICIPANT
 - dbo.RESULT
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
 - Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler

SQLQuery5 cr...\\clive (52))*

```

41
42
43 /* =====
44 4. CREATE EVENT TABLE
45 ===== */
46
47 CREATE TABLE EVENT
48 (
49     EventID INT IDENTITY(1,1) PRIMARY KEY,
50     EventName VARCHAR(100) NOT NULL,
51     Description VARCHAR(500) NOT NULL,
52     EventDate DATE NOT NULL,
53     Location VARCHAR(150) NOT NULL,
54     Distance DECIMAL(6,2) NOT NULL,
55     EventType VARCHAR(20) NOT NULL,
56     OrganizerID INT NOT NULL,
57
58     CONSTRAINT CK_EVENT_Distance
59         CHECK (Distance > 0),
60
61     CONSTRAINT CK_EVENT_EventType
62         CHECK (EventType IN ('Run', 'Walk', 'Cycle')),
63
64     CONSTRAINT FK_EVENT_Organizer
65         FOREIGN KEY (OrganizerID)
66         REFERENCES ORGANIZER(OrganizerID)
67 );
68
69
70 /* =====
71 5. CREATE CATEGORY TABLE
72 ===== */
73
74 CREATE TABLE CATEGORY
75 (
76     CategoryID INT IDENTITY(1,1) PRIMARY KEY,
77     CategoryName VARCHAR(60) NOT NULL,
78     CategoryType VARCHAR(20) NOT NULL,
79     EventID INT NOT NULL,
80
81     CONSTRAINT CK_CATEGORY_CategoryType
82         CHECK (CategoryType IN ('Age', 'Distance')),
83
84     CONSTRAINT FK_CATEGORY_Event
85         FOREIGN KEY (EventID)
86         REFERENCES EVENT(EventID)
87 );
88
89
90 /* =====
91 6. CREATE ENROLMENT TABLE
92 ===== */
93
94 CREATE TABLE ENROLMENT

```

Object Explorer

Connect

SQLSERVER (SQL Server 16.0.1000)

- Databases
 - System Databases
 - Database Snapshots
 - ArtShowDB
 - RaceDayDB
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.CATEGORY
 - dbo.ENROLMENT
 - dbo.EVENT
 - dbo.ORGANIZER
 - dbo.PARTICIPANT
 - dbo.RESULT
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
 - Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler

SQLQuery5 cr...\clive (52))*

```

89
90 /* =====
91 6. CREATE ENROLMENT TABLE
92 ===== */
93
94 CREATE TABLE ENROLMENT
95 (
96     EnrolmentID INT IDENTITY(1,1) PRIMARY KEY,
97     EnrolmentDate DATE NOT NULL
98         DEFAULT GETDATE(),
99     Status VARCHAR(20) NOT NULL
100         DEFAULT 'Confirmed',
101     ParticipantID INT NOT NULL,
102     EventID INT NOT NULL,
103     CategoryID INT NOT NULL,
104
105     CONSTRAINT CK_ENROLMENT_Status
106         CHECK (Status IN ('Pending', 'Confirmed', 'Cancelled')),
107
108     CONSTRAINT FK_ENROLMENT_Participant
109         FOREIGN KEY (ParticipantID)
110         REFERENCES PARTICIPANT(ParticipantID),
111
112     CONSTRAINT FK_ENROLMENT_Event
113         FOREIGN KEY (EventID)
114         REFERENCES EVENT(EventID),
115
116     CONSTRAINT FK_ENROLMENT_Category
117         FOREIGN KEY (CategoryID)
118         REFERENCES CATEGORY(CategoryID),
119
120     CONSTRAINT UQ_ENROLMENT_Participant_Event
121         UNIQUE (ParticipantID, EventID)
122 );
123
124 /* =====
125 7. CREATE RESULT TABLE
126 ===== */
127
128 CREATE TABLE RESULT
129 (
130     ResultID INT IDENTITY(1,1) PRIMARY KEY,
131     FinishTime TIME NOT NULL,
132     Position INT NOT NULL,
133     EnrolmentID INT NOT NULL UNIQUE,
134
135     CONSTRAINT CK_RESULT_Position
136         CHECK (Position > 0),
137
138     CONSTRAINT FK_RESULT_Enrolment
139         FOREIGN KEY (EnrolmentID)
140         REFERENCES ENROLMENT(EnrolmentID)
141 );
142
  
```

SQLQuery2.sql...\clive (69))*

Insert Statements

The screenshot displays the SQL Server Enterprise Manager interface. The Object Explorer on the left shows the database structure for 'RaceDayDB', including tables like 'dbo.CATEGORY', 'dbo.ENROLMENT', 'dbo.EVENT', 'dbo.ORGANIZER', 'dbo.PARTICIPANT', and 'dbo.RESULT'. The main query window, titled 'SQLQuery5 cr...\clive (52))*', contains the following SQL code:

```
125  /* =====
126  7. CREATE RESULT TABLE
127  ===== */
128
129  CREATE TABLE RESULT
130  (
131      ResultID INT IDENTITY(1,1) PRIMARY KEY,
132      FinishTime TIME NOT NULL,
133      Position INT NOT NULL,
134      EnrolmentID INT NOT NULL UNIQUE,
135
136      CONSTRAINT CK_RESULT_Position
137          CHECK (Position > 0),
138
139      CONSTRAINT FK_RESULT_Enrolment
140          FOREIGN KEY (EnrolmentID)
141              REFERENCES ENROLMENT(EnrolmentID)
142  );
143
144
145  /* =====
146  8. INSERT ORGANISERS
147      Minimum required: 2
148  ===== */
149
150  INSERT INTO ORGANIZER
151      (FirstName, LastName, Email, PasswordHash)
152  VALUES
153      ('Thabo', 'Wilson', 'thabo.wilson@gmail.com', 'hashed_password_001'),
154      ('Sarah', 'Jones', 'srah_jones@gmail.com', 'hashed_password_002');
155
156
157  Verification query
158
159  SELECT *
160  FROM ORGANIZER;
161
162  /* =====
163  9. INSERT PARTICIPANTS
164      Minimum required: 2
165  ===== */
166
167  INSERT INTO PARTICIPANT
168      (FirstName, LastName, Email, PasswordHash)
169  VALUES
170      ('Jessica', 'Woods', 'jess.woodz@outlook.com', 'hashed_password_003'),
171      ('Lerato', 'Dlamini', 'lerato.dlamini@gmail.com', 'hashed_password_004');
172
173
174  Verification query
175
176  SELECT *
177  FROM PARTICIPANT;
```

The Results pane at the bottom shows the data inserted into the Organizer table:

	OrganizerID	FirstName	LastName	Email	PasswordHash
1	1	Thabo	Wilson	thabo.wilson@gmail.com	hashed_password_001
2	2	Sarah	Jones	srah_jones@gmail.com	hashed_password_002

Results		Messages			
	ParticipantID	FirstName	LastName	Email	PasswordHash
1	1	Jessica	Woods	jess.woodz@outlook.com	hashed_password_003
2	2	Lerato	Dlamini	lerato.dlamini@gmail.com	hashed_password_004

Object Explorer

Connect

SQL Server 16.0.1000

Databases

System Databases

Database Snapshots

ArtShowDB

RaceDayDB

Database Diagrams

Tables

System Tables

FileTables

External Tables

Graph Tables

dbo.CATEGORY

dbo.ENROLMENT

dbo.EVENT

dbo.ORGANIZER

dbo.PARTICIPANT

dbo.RESULT

Views

External Resources

Synonyms

Programmability

Query Store

Service Broker

Storage

Security

Security

Server Objects

Replication

Management

XEvent Profiler

Execute

SQLQuery5 cr...\clive (52))*

SQLQuery2.sql...\clive (69))*

```

161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219

```

```

/* =====
9. INSERT PARTICIPANTS
Minimum required: 2
===== */

INSERT INTO PARTICIPANT
(FirstName, LastName, Email, PasswordHash)
VALUES
('Jessica', 'Woods', 'jess.woodz@outlook.com', 'hashed_password_003'),
('Lerato', 'Dlamini', 'lerato.dlamini@gmail.com', 'hashed_password_004');

Verification query

SELECT *
FROM PARTICIPANT;

/* =====
10. INSERT EVENTS
Minimum required: 3
===== */

INSERT INTO EVENT
(EventName, Description, EventDate, Location, Distance, EventType, OrganizerID)
VALUES
(
    'Pretoria City Run',
    'A 10 kilometre road running event through Pretoria.',
    '2026-11-10',
    'Pretoria',
    10.00,
    'Run',
    1
),
(
    'Tshwane Family Walk',
    'A community walking event suitable for families.',
    '2026-10-17',
    'Tshwane',
    5.00,
    'Walk',
    1
),
(
    'Gauteng Cycle Challenge',
    'A competitive cycling event for recreational and experienced cyclists.',
    '2026-11-07',
    'Johannesburg',
    40.00,
    'Cycle',
    2
);

Verification query

SELECT *
FROM EVENT;

```

Object Explorer

Connect

SQL Server 16.0.1000

Databases

- System Databases
- Database Snapshots
- ArtShowDB
- RaceDayDB
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.CATEGORY
 - dbo.ENROLMENT
 - dbo.EVENT
 - dbo.ORGANIZER
 - dbo.PARTICIPANT
 - dbo.RESULT
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
- Security
- Server Objects
- Replication
- Management
- XEvent Profiler

SQLQuery5 cr...\clive (52))*

```

215
216 Verification query
217
218 SELECT *
219 FROM EVENT;
220
221 /* =====
222 11. INSERT CATEGORIES
223 Each event receives multiple categories.
224 ===== */
225
226 INSERT INTO CATEGORY
227 (CategoryName, CategoryType, EventID)
228 VALUES
229 ('Under 20', 'Age', 1),
230 ('Senior', 'Age', 1),
231 ('10km', 'Distance', 1),
232
233 ('Under 20', 'Age', 2),
234 ('Senior', 'Age', 2),
235 ('5km', 'Distance', 2),
236
237 ('Junior', 'Age', 3),
238 ('Senior', 'Age', 3),
239 ('40km', 'Distance', 3);
240
241 Verification query
242
243 SELECT
244 C.CategoryID,
245 C.CategoryName,
246 C.CategoryType,
247 E.EventName
248 FROM CATEGORY C
249 INNER JOIN EVENT E
250 ON C.EventID = E.EventID;
251
252 /* =====
253 12. INSERT ENROLMENTS
254 ===== */
255
256 INSERT INTO ENROLMENT
257 (EnrolmentDate, Status, ParticipantID, EventID, CategoryID)
258 VALUES
259 ('2026-09-01', 'Confirmed', 1, 1, 1),
260 ('2026-09-02', 'Confirmed', 2, 1, 2),
261 ('2026-09-03', 'Confirmed', 1, 2, 4),
262 ('2026-09-04', 'Confirmed', 2, 2, 5),
263 ('2026-09-05', 'Confirmed', 1, 3, 7),
264 ('2026-09-06', 'Confirmed', 2, 3, 8);
265
266 Verification query
267
268 SELECT
269 EN.EnrolmentID,
270 P.FirstName + ' ' + P.LastName AS Participant,
271 E.EventName,
272

```

SQLQuery2.sql...\clive (69))*

Results		Messages		
	CategoryID	CategoryName	CategoryType	EventName
1	1	Under 20	Age	Pretoria City Run
2	2	Senior	Age	Pretoria City Run
3	3	10km	Distance	Pretoria City Run
4	4	Under 20	Age	Tshwane Family Walk
5	5	Senior	Age	Tshwane Family Walk
6	6	5km	Distance	Tshwane Family Walk
7	7	Junior	Age	Gauteng Cycle Challenge
8	8	Senior	Age	Gauteng Cycle Challenge
9	9	40km	Distance	Gauteng Cycle Challenge

67 %		No issues found			
Results		Messages			
	EnrolmentID	Participant	EventName	CategoryName	Status
1	1	Jessica Woods	Pretoria City Run	Under 20	Confirmed
2	2	Lerato Dlamini	Pretoria City Run	Senior	Confirmed
3	3	Jessica Woods	Tshwane Family Walk	Under 20	Confirmed
4	4	Lerato Dlamini	Tshwane Family Walk	Senior	Confirmed
5	5	Jessica Woods	Gauteng Cycle Challenge	Junior	Confirmed
6	6	Lerato Dlamini	Gauteng Cycle Challenge	Senior	Confirmed

Object Explorer

Connect

SQLServer Express (SQL Server 16.0.1000)

- Databases
 - System Databases
 - Database Snapshots
 - ArtShowDB
 - RaceDayDB
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.CATEGORY
 - dbo.ENROLMENT
 - dbo.EVENT
 - dbo.ORGANIZER
 - dbo.PARTICIPANT
 - dbo.RESULT
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
 - Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler

SQLQuery5 cr...\clive (52))*

```

266
267
268 Verification query
269
270 SELECT
271     EN.EnrolmentID,
272     P.FirstName + ' ' + P.LastName AS Participant,
273     E.EventName,
274     C.CategoryName,
275     EN.Status
276 FROM ENROLMENT EN
277 INNER JOIN PARTICIPANT P
278     ON EN.ParticipantID = P.ParticipantID
279 INNER JOIN EVENT E
280     ON EN.EventID = E.EventID
281 INNER JOIN CATEGORY C
282     ON EN.CategoryID = C.CategoryID;
283
284 /*
285 13. INSERT RESULTS
286 */
287
288 INSERT INTO RESULT
289     (FinishTime, Position, EnrolmentID)
290 VALUES
291     ('00:48:35', 1, 1),
292     ('00:52:14', 2, 2),
293     ('01:05:20', 1, 3),
294     ('01:12:45', 2, 4);
295
296 Verification query
297
298 SELECT
299     R.ResultID,
300     P.FirstName + ' ' + P.LastName AS Participant,
301     E.EventName,
302     R.FinishTime,
303     R.Position
304 FROM RESULT R
305 INNER JOIN ENROLMENT EN
306     ON R.EnrolmentID = EN.EnrolmentID
307 INNER JOIN PARTICIPANT P
308     ON EN.ParticipantID = P.ParticipantID
309 INNER JOIN EVENT E
310     ON EN.EventID = E.EventID;
311
312
313
314
  
```

67 % No issues found

Results Messages

	ResultID	Participant	EventName	FinishTime	Position
1	1	Jessica Woods	Pretoria City Run	00:48:35.0000000	1
2	2	Lerato Dlamini	Pretoria City Run	00:52:14.0000000	2
3	3	Jessica Woods	Tshwane Family Walk	01:05:20.0000000	1
4	4	Lerato Dlamini	Tshwane Family Walk	01:12:45.0000000	2

The SQL database was built following the structure shown in the ERD. Six tables were created: Organizer, Participant, Event, Category, Enrolment, and Result. Primary key columns were added to each table to uniquely identify each record. Foreign key columns were added to link the tables. Additional constraints such as NOT NULL, UNIQUE, DEFAULT, and CHECK were added to keep the data clean and to stop data from being entered.

The Event table holds an EventType column, and a CHECK constraint limits the value of EventType to Run, Walk or Cycle. The Enrolment table holds keys to Participant, Event and Category so the system can record which participant joined which event and which category was chosen. The Result table refers to Enrolment so that finishing data is linked to the event participation. Sample data was inserted into Organizer, Participant, Event, Category, Enrolment and Result tables. Verification queries were run to confirm that the data was inserted correctly and that the relationships between tables work as expected.